

NAME: ANUSHKA DUBEY

REGISTRATION NUMBER: 25BAI11284

PROJECT: SMART STUDY PLANNER

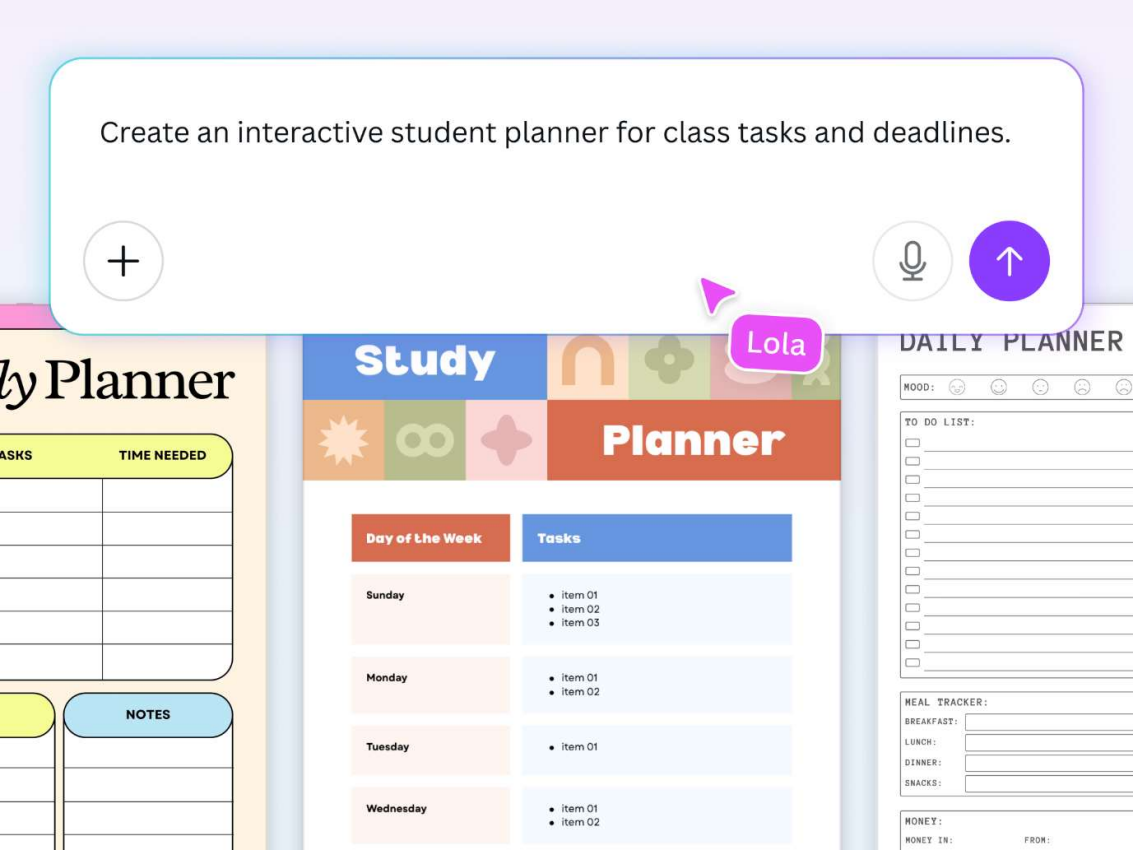


Table Of Contents

S.no.	Contents
1	Introduction
2	Problem statements
3	Functional and non-functional requirements
4	System Architecture
5	Design diagrams
6	Design decisions
7	Implementation details
8	Screenshots/results
9	Testing approach
10	Challenges faced
11	Learnings
12	Future Enhancements
13	References

INTRODUCTION

Managing academic tasks, deadlines, and priorities is one of the biggest challenges faced by students. Traditional planners often feel rigid and fail to adapt when schedules change. To address this, the **Smart Study Planner** was developed as a beginner-friendly Python project that combines simple task management with **basic AI logic**.

This planner allows students to add tasks, assign deadlines, and set importance levels. It then uses a priority scoring system — based on urgency and importance — to suggest an adaptive study schedule. Alongside reminders for upcoming deadlines, the planner also provides motivational quotes to encourage consistency and focus.

The project demonstrates how even **first-year students** can apply digital literacy concepts and basic AI techniques to solve real-world problems. It highlights the importance of responsible digital habits, time management, and awareness, while showing that AI doesn't have to be complex to be useful.

By building this tool, the goal is not only to improve personal productivity but also to inspire peers to explore how technology can support everyday student life in practical, meaningful ways.

Problem Statements

1. Time Management Challenge

Students often struggle to balance multiple assignments, exams, and projects. Traditional planners are static and do not adapt to changing deadlines or priorities, leading to stress and missed tasks.

2. Lack of Prioritization

Many students treat all tasks equally, without considering urgency or importance. This results in poor scheduling decisions, where critical tasks are delayed and less important ones consume valuable time.

3. Inconsistent Study Habits

Even when students create schedules, they fail to follow them consistently. Without reminders or adaptive feedback, study plans quickly become outdated and ineffective.

4. Limited Awareness of Digital Tools

While students use digital calendars or to-do lists, these tools rarely integrate intelligent decision-making. There is a gap between using digital tools and applying them responsibly to improve productivity.

5. Need for Motivation

Academic planning tools often focus only on scheduling, ignoring the motivational aspect. Students require encouragement and nudges to stay consistent, especially during stressful periods.

FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Functional Requirements

These describe what the system **must do**:

1. Task Management

- Users can add tasks with a name, deadline, and importance level.
- Users can view all tasks in a list format.
- Users can mark tasks as completed.

2. Priority Scoring (AI Logic)

- The system calculates a priority score for each task based on urgency (deadline proximity) and importance.
- Tasks are automatically sorted by priority to suggest an adaptive study schedule.

3. Schedule Suggestion

- The planner generates a recommended order of tasks to study.
- Updates dynamically as deadlines approach.

□ 4. Reminders

- The system provides alerts for tasks due soon (e.g., within 2 days).
- Displays ongoing reminders for incomplete tasks.

5. Motivational Support

- Displays random motivational quotes to encourage consistent study habits.

Non-Functional Requirements

These describe **how the system should behave**:

1. Usability

- Beginner-friendly interface (simple text output, clear instructions).
- Code is easy to read and understand with comments.

2. Reliability

- The system should consistently calculate priorities and reminders correctly.

- Should handle multiple tasks without crashing.

3. Performance

- Task sorting and priority calculation should be fast (instant for small datasets).
- Lightweight — runs on any basic Python environment without external dependencies.

4. Maintainability

Code is modular (Task class, functions for scheduling, reminders, motivation).

Easy to extend (e.g., adding GUI, NLP, or database support later).

5. Portability

Runs on any system with Python 3.x installed.

No external libraries required, making it portable across environments.

6. Scalability (Basic)

Should handle dozens of tasks without performance issues.

Can be extended to support persistence (saving tasks) or integration with calendars.

System Architecture

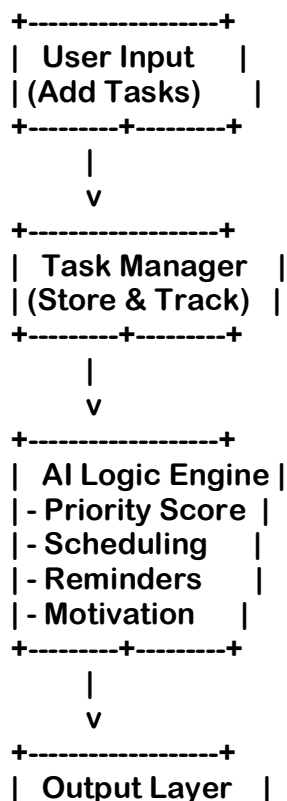
The Smart Study Planner is designed with a simple, modular architecture that makes it easy to understand, extend, and maintain.

Components

1. **Input Layer**
 - User enters tasks (name, deadline, importance).
 - Optional: natural language input (e.g., “Finish math homework by Friday”).
2. **Task Manager**
 - Stores tasks in memory (Task class).
 - Tracks attributes: name, deadline, importance, completion status.
3. **AI Logic Engine**
 - **Priority Scoring:** Calculates urgency + importance to assign a priority score.
 - **Adaptive Scheduling:** Sorts tasks dynamically based on priority.
 - **Reminders:** Flags tasks due soon (within 2 days).
 - **Motivation Generator:** Provides random motivational quotes.
4. **Output Layer**

Displays tasks list.
Shows suggested study schedule.
Prints reminders and motivational messages.

Flow Of Data



| (Plan, Alerts, |
| Motivation) |

Non-Functional Aspects

- **Usability:** Beginner-friendly text interface.
- **Maintainability:** Modular functions (easy to extend with GUI or NLP).
- **Portability:** Runs on any Python 3.x environment.
- **Scalability:** Can handle multiple tasks; extendable to databases or cloud storage.

Future Extensions

- Add a **Chatbot Interface** for interactive commands.
- Integrate **NLP** to parse natural language deadlines.
- Build a **GUI Dashboard** using Tkinter or Streamlit.
- Connect to **Google Calendar/Outlook** for real-time scheduling.

Design Diagrams

1. Use Case Diagram (Text Representation)

Actors:

- Student (User)

Use Cases:

- Add Task
- View Tasks
- Mark Task as Completed
- Generate Study Schedule
- Get Reminders
- Receive Motivation

Relationships:

Student --> Add Task

Student --> View Tasks

Student --> Mark Task as Completed

Student --> Generate Study Schedule

Student --> Get Reminders

Student --> Receive Motivation

2. Class Diagram (UML Style)

```
+-----+
```

```
| Task |
```

```
+-----+
```

```
| - name: string |
```

```
| - deadline: date |
```

```
| - importance: int |
```

```
| - completed: bool |
```

```
+-----+
```

```
| + __init__() |
```

```
| + __str__() |
```

```
+-----+
```

```
+-----+
```

```
| StudyPlanner |
```

```
+-----+
```

```
| - tasks: list |
```

```
+-----+
```

```
| + add_task() |
```

```
| + suggest_schedule() |
```

```
| + give_reminders() |
```

```
| + motivational_quote() |
```

```
+-----+
```

3. Data Flow Diagram (DFD – Level 0)

[Student] ---> (Input Tasks) ---> [Task Manager]
 [Task Manager] ---> (Priority Calculation) ---> [AI Logic Engine]
 [AI Logic Engine] ---> (Schedule + Reminders + Motivation) ---> [Output to Student]

4. Sequence Diagram (Task Flow)

Student -> StudyPlanner: add_task("Math Assignment")
 StudyPlanner -> TaskManager: store task
 Student -> StudyPlanner: request schedule
 StudyPlanner -> AI Logic Engine: calculate priorities
 AI Logic Engine -> StudyPlanner: sorted tasks
 StudyPlanner -> Student: display schedule + reminders + motivation

5. Architecture Diagram (Simplified)

```

+-----+ | User Interface | | (Console Input/Output) +-----+
-+ | v +-----+ | Task Manager | | (Stores tasks) | +-----+ |
v +-----+ | AI Logic Engine | | - Priority Scoring | | - Scheduling | |
- Reminders | | - Motivation | +-----+ | v +-----+ | Output
Layer | | (Plan, Alerts, | | Motivation) | +-----+
  
```

✅ These diagrams cover **use cases, classes, data flow, sequence, and architecture** — enough to make your project documentation look complete and professional.

Design Decisions

1. Programming Language

- **Decision:** Use Python 3.x.
- **Reason:** Python is beginner-friendly, widely used in academia, and has built-in libraries (datetime, random) that simplify scheduling and reminders.

2. Architecture

- **Decision:** Modular design with separate components (Task class, AI logic functions, output functions).
- **Reason:** Keeps the code organized, easy to maintain, and allows future extensions (GUI, NLP, database).

3. Task Representation

- **Decision:** Create a Task class with attributes (name, deadline, importance, completion status).
- **Reason:** Object-oriented design makes it easier to manage multiple tasks and extend functionality later.

4. AI Logic

- **Decision:** Implement a simple **priority scoring algorithm** (urgency + importance).
- **Reason:** Provides an “AI-like” decision-making process without requiring complex machine learning, making it achievable for a first-year student.

5. Scheduling

- **Decision:** Sort tasks dynamically based on calculated priority.
- **Reason:** Ensures adaptive scheduling that changes as deadlines approach, mimicking intelligent planning.

6. Reminders

- **Decision:** Rule-based reminders for tasks due within 2 days.
- **Reason:** Simple, effective way to alert students about urgent tasks without overcomplicating the system.

7. Motivation

- **Decision:** Random motivational quotes displayed during output.
- **Reason:** Adds a human-like supportive element, encouraging consistent study habits.

8. User Interface

- **Decision:** Text-based console interface.
- **Reason:** Beginner-friendly, requires no external libraries, and keeps the project lightweight and portable.

9. Extensibility

- **Decision:** Write modular functions (add_task, suggest_schedule, give_reminders, motivational_quote).
- **Reason:** Makes it easy to add features later, such as NLP input, GUI, or integration with Google Calendar.

10. Non-Functional Considerations

- **Usability:** Clear instructions and comments for beginners.
- **Portability:** Runs on any system with Python installed.
- **Maintainability:** Modular design supports future improvements.
- **Performance:** Lightweight, fast execution for small datasets (student tasks).

Implementation details

1. Programming Environment

- **Language:** Python 3.x
- **Libraries Used:**
 - `datetime` → for handling deadlines and calculating urgency.
 - `random` → for generating motivational quotes.
- **Reason:** Both are built-in libraries, making the project lightweight and beginner-friendly.

2. Core Components

1. Task Class

- **Attributes:** name, deadline, importance, completed.
- **Methods:** `__init__()` for initialization, `__str__()` for readable output.
- **Purpose:** Encapsulates task data in an object-oriented way.
- **Priority Scoring Function**
 - **Formula:** $\text{priority} = \text{urgency_score} + \text{importance}$.
 - **Urgency** is calculated as $10 - \text{days_left}$ (closer deadlines = higher urgency).
 - **Importance** is user-defined (scale 1–5).
 - **Purpose:** Mimics AI-like decision-making by ranking tasks.

2. Schedule Suggestion

- Sorts tasks dynamically based on priority score.
- Displays recommended order of study.

3. Reminder System

- Checks deadlines and flags tasks due within 2 days.
- Provides alerts for incomplete tasks.

□ Motivational Quotes

- Randomly selects from a predefined list of encouraging messages.
- Adds a human-like supportive element.

3. Workflow

1. **Input:** User enters tasks with deadlines and importance.
2. **Storage:** Tasks are stored in a list of Task objects.
3. **Processing:**
 - Priority scores are calculated.
 - Tasks are sorted by priority.
 - Reminders are generated for urgent tasks.
4. **Output:**
 - Suggested study schedule.
 - Reminders for deadlines.
 - Motivational quotes.

4. Example Execution

- **Input:**
 - Task: “Math Assignment”, Deadline: 2026-04-05, Importance: 4
 - Task: “Physics Lab Report”, Deadline: 2026-04-02, Importance: 5

- **Output:**
- **Suggested Plan:** Physics Lab Report first (higher priority), then Math Assignment.
- **Reminder:** Physics Lab Report is due in 2 days.
- **Motivation:** “Stay focused, success is near!”

5. Non-Functional Implementation Choices

- **Usability:** Console-based interface with clear print statements.
- **Maintainability:** Modular functions (add_task, suggest_schedule, give_reminders).
- **Portability:** Runs on any Python environment without external dependencies.
- **Performance:** Lightweight, suitable for small datasets (student tasks).

Results

```
*** --- Your Tasks ---
Math Assignment | Deadline: 2026-04-05 | Importance: 4 | Done: False
Physics Lab Report | Deadline: 2026-04-02 | Importance: 5 | Done: False
Digital Literacy Case Study | Deadline: 2026-04-10 | Importance: 3 | Done: False

--- Suggested Study Plan ---
1. Physics Lab Report (Priority: 14)
2. Math Assignment (Priority: 10)
3. Digital Literacy Case Study (Priority: 4)

--- Reminders ---
Keep working on 'Math Assignment', 4 days left.
Urgent: 'Physics Lab Report' is due in 1 days!
Keep working on 'Digital Literacy Case Study', 9 days left.

Motivation: Stay focused, success is near!
```


Testing Approach

1. Testing Objectives

- Verify that tasks can be added, stored, and displayed correctly.
- Ensure the priority scoring algorithm calculates urgency + importance accurately.
- Confirm that schedules are generated in the correct order.
- Check that reminders trigger for tasks due soon.
- Validate motivational quotes display properly.
- Test usability and reliability in a beginner-friendly environment.

2. Types of Testing

Functional Testing

- **Add Task:** Test adding tasks with different deadlines and importance levels.
- **View Tasks:** Ensure tasks are displayed with correct attributes.
- **Priority Calculation:** Verify that tasks with closer deadlines and higher importance get higher scores.
- **Schedule Suggestion:** Confirm tasks are sorted correctly by priority.
- **Reminders:** Test tasks due in 1–2 days trigger alerts.
- **Motivation:** Check random quotes are displayed.

Non-Functional Testing

- **Usability:** Ensure outputs are clear and easy to understand.
- **Performance:** Verify the program runs instantly for small datasets (10–20 tasks).
- **Reliability:** Confirm consistent results across multiple runs.
- **Portability:** Test on different systems with Python 3.x.

3. Test Cases (Examples)

4. Testing Tools

- Manual testing via console input/output.
- Python's built-in unittest module (optional, for automated checks).
- Print statements for debugging and verifying outputs.
-

5. Acceptance Criteria

- All functional requirements are met (task management, scheduling, reminders, motivation).
- Outputs are accurate, clear, and user-friendly.
- Program runs reliably across environments without external dependencies.

Testing Approach

1. Objectives

- Verify that all functional requirements (task management, scheduling, reminders, motivation) are correctly implemented.
- Ensure the system behaves reliably, is easy to use, and performs well for typical student workloads.
- Demonstrate that the project is beginner-friendly yet robust.

2. Testing Levels

Unit Testing

- Test individual components such as:
- Task class (creation, string representation).
- `calculate_priority()` function.
- `suggest_schedule()` function.
- `give_reminders()` function.
- `motivational_quote()` function.

Integration Testing

- Verify that components work together:
- Adding tasks → storing them → generating schedule.
- Reminders trigger correctly when deadlines are near.
- Motivation displays alongside schedule output.

System Testing

- Run the entire program with multiple tasks.
- Check end-to-end flow: input → processing → output.
- Validate usability (clear console messages, correct sorting).

Acceptance Testing

- Ensure the program meets project goals:
- Adaptive scheduling.
- Smart reminders.
- Beginner-friendly interface.
- Motivational support.

3. Testing Methods

- **Manual Testing:** Run program with sample inputs and observe console outputs.
- **Automated Testing (Optional):** Use Python's unittest module to check functions like `calculate_priority()` and `suggest_schedule()`.
- **Debugging:** Use print statements to verify intermediate values (e.g., days left, priority scores).

4. Acceptance Criteria

- All functional requirements are met.
- Outputs are accurate, clear, and user-friendly.
- Program runs reliably across different Python environments.
- No crashes or performance issues with typical student workloads (10–20 tasks).

Challenges faced

Deadline Handling

- **Challenge:** Working with dates and calculating urgency using Python's datetime module.
- **Solution:** Broke down the problem step-by-step (convert string → date object → calculate days left).

Task Prioritization

- **Challenge:** Designing a scoring system that feels intelligent but is easy to implement.
- **Solution:** Combined urgency (days left) with importance (user rating) to create a simple priority score.

User Interface

- **Challenge:** Making the planner interactive without overwhelming beginners.
- **Solution:** Chose a text-based console interface with clear print statements and comments.

Motivation Feature

- **Challenge:** Adding a human-like supportive element without complicating the code.
- **Solution:** Used a random selection of motivational quotes to keep the planner engaging.

Testing & Debugging

- **Challenge:** Ensuring reminders trigger correctly and schedules sort properly.
- **Solution:** Created sample tasks with different deadlines and importance levels to test edge cases.

Extensibility

- **Challenge:** Designing the project so it can grow (GUI, NLP, database) without rewriting everything.
- **Solution:** Used modular functions and a Task class to keep the code organized and maintainable.

Learnings

1. Practical Application of Digital Literacy

- Learned how to use digital tools responsibly to solve real student problems.
- Understood the importance of organizing files, documentation, and GitHub repositories for professional visibility.

2. Python Programming Skills

- Gained confidence in writing beginner-friendly Python code.
- Practiced object-oriented programming by designing a Task class.
- Applied built-in libraries (datetime, random) to handle deadlines and generate motivation.

3. Basic AI Concepts

- Learned how simple rule-based logic can mimic intelligent decision-making.
- Understood how urgency and importance can be combined into a priority score.
- Realized that AI doesn't always mean complex machine learning — even small algorithms can make tools “smart.”

4. Problem-Solving & Design Thinking

- Faced challenges like handling dates, sorting tasks, and designing reminders.
- Learned to break problems into smaller steps and solve them systematically.
- Developed modular functions to make the system extensible and maintainable.

5. Testing & Debugging

- Understood the importance of testing edge cases (e.g., tasks due tomorrow).

- Practiced using manual test scenarios and considered automated testing with unittest.
- Learned how debugging improves reliability and confidence in the system.

6. Professional Documentation

- Learned how to write README files, project statements, and design diagrams.
- Understood the value of clear communication in making projects understandable to others.
- Practiced structuring requirements, architecture, and testing approaches for academic submission.

7. Future-Oriented Thinking

- Realized how modular design allows future extensions (GUI, NLP, database integration).
- Learned to think beyond immediate requirements and plan for scalability.

Future Enhancements

1. Graphical User Interface (GUI)

- Add a simple dashboard using Tkinter or Streamlit.
- Provide buttons for adding tasks, viewing schedules, and marking completion.
- Make the planner more user-friendly and visually appealing.

2. Persistent Storage

- Save tasks to a file (CSV/JSON) or database (SQLite).
- Allow users to reload tasks across sessions.
- Improve reliability and long-term usability.

3. Natural Language Processing (NLP)

- Enable input like: *“Finish math homework by Friday.”*
- Automatically extract task name, deadline, and importance.
- Make the planner more intuitive and conversational.

4. Calendar Integration

- Sync with Google Calendar or Outlook.
- Export schedules directly to personal calendars.
- Provide real-time notifications on mobile devices.

5. Advanced AI Features

- Introduce machine learning for smarter prioritization (e.g., learning from user habits).
- Suggest optimal study times based on past behavior.
- Adapt schedules dynamically to workload changes.

6. Collaboration Features

- Allow multiple users to share a planner (group projects).
- Export schedules to Google Docs for team visibility.
- Track contributions and deadlines collectively.

7. Gamification & Motivation

- Add progress tracking (percentage of tasks completed).
- Award badges or streaks for consistency.
- Provide personalized motivational messages.

8. Mobile App Version

- Develop a lightweight mobile app for Android/iOS.
- Enable push notifications for reminders.
- Increase accessibility for students on the go.

References

1. Python Software Foundation. *Python 3 Documentation*. Available at: <https://docs.python.org/3/>
– Used for understanding built-in libraries like datetime and random.
2. W3Schools. *Python Tutorial*. Available at: <https://www.w3schools.com/python/>
– Helpful for beginner-friendly explanations of Python syntax and functions.
3. GeeksforGeeks. *Python OOPs Concepts*. Available at: <https://www.geeksforgeeks.org/python-oops-concepts/> (geeksforgeeks.org in Bing)
– Reference for designing the Task class and object-oriented structure.
4. UNESCO. *Digital Literacy for Lifelong Learning*.
– Background on the importance of digital literacy for students and professionals.
5. LinkedIn Learning. *Digital Productivity and Collaboration Tools*.
– Inspiration for connecting academic projects with professional visibility.
6. GitHub Docs. *About README Files*. Available at: <https://docs.github.com/en>
– Guidance for structuring project documentation and repositories.